RCMM — Release Candidate Management Module

RCMM — Release Candidate Management Module

OOF® Origin Open Foundation®

Independent Methodological Authority

Architecture: SLGA™ — Software Lifecycle Governance Architecture

Parent Standard: SRLS — Software Release Standard

Category: Governance & Enforcement

Subcategory: Software Lifecycle Governance

Type: Core Module

Governed Space: Software Release Candidate Governance

Module ID: SLGA-SRLS-RCMM-01

Origin ID: OOF-SLGA-SRLS-RCMM

Version: 1.0

Status: Canonical Open

Effective Date: 13.10.2026

Compatibility: Methodology OS — Modular Compatibility

GIE-Compatible: Yes

AI-Readable: Yes

Authority: OOF® — OriginOpen® Foundation

Protection: Protected Methodological Work

Canonical Language: English

Version: ### 7\. Scope

Version: A candidate identifier must refer to a sufficiently defined candidate state

Version: ### 43\. Evidence Integrity

Version: ### 62\. Minimum Implementation Framework

Version: ### 69\. Practical Applications

RCMM — Release Candidate Management Module

Architecture

SLGA™ — Software Lifecycle Governance Architecture

Parent Standard

SRLS — Software Release Standard

Category

Governance & Enforcement

Subcategory

Software Lifecycle Governance

Type

Core Module

Governed Space

Software Release Candidate Governance

Module ID

SLGA-SRLS-RCMM-01

Origin ID

OOF-SLGA-SRLS-RCMM

Version

1.0

Status

Canonical Open

Effective Date

10.2026

Compatibility

Methodology OS — Modular Compatibility

GIE-Compatible

Yes

AI-Readable

Yes

Authority

OOF® — OriginOpen® Foundation

Protection

Protected Methodological Work

Canonical Language

English

1\. Canonical Definition

RCMM — Release Candidate Management Module is the Core Module within SRLS that governs how software artifacts are identified, assembled, distinguished, bounded, registered, maintained, and presented as release candidates for subsequent release readiness assessment and authorization.

RCMM establishes the methodological conditions under which a software

artifact or defined collection of artifacts may be recognized as an identifiable release candidate with a sufficiently established composition, configuration, dependency context, intended scope, and supporting lifecycle references.

It governs the relationship between the software proposed for release and

the exact artifacts, builds, configurations, components, and conditions represented by that candidate.

RCMM ensures that release candidates remain distinguishable from development versions, intermediate builds, alternative configurations, previously authorized releases, and deployed software states.

It preserves candidate identity and traceability when artifacts are generated, assembled, modified, replaced, or submitted through human-operated, automated, or autonomous software development processes.

The module does not independently establish release readiness or grant release authorization.

Its principal governed output is the Software Release Candidate Profile.

Software Artifact Available ≠ Release Candidate Established.

Version Label Assigned ≠ Release Candidate Identity Verified.

Release Candidate Identified ≠ Release Candidate Ready.

Release Candidate Registered ≠ Release Authorized.

2\. Operational Role

RCMM governs the identification and controlled preparation of software

release candidates.

Its operational role is to establish:

Release candidate identity.

Candidate purpose and intended release scope.

Software product and system association.

Candidate composition.

Source artifact references.

Build artifact references.

Configuration baseline references.

Dependency baseline references.

Candidate version and revision.

Artifact integrity references.

Candidate assembly context.

Candidate creation and modification events.

Participating actor references.

Automated creation context.

Candidate completeness.

Candidate boundaries and exclusions.

Candidate evidence sufficiency.

Candidate changes and supersession.

Candidate status and history.

Readiness assessment handoff.

RCMM provides the identifiable candidate upon which subsequent SRLS release

governance depends.

It does not independently perform quality testing, security assessment, release authorization, deployment execution, or operational recovery.

3\. Operational Question

How can software proposed for release be established as an identifiable, bounded, compositionally traceable, and evidence-supported release candidate when its artifacts, builds, dependencies, configurations, creators, and automated development processes may change before release readiness and authorization are determined?

4\. Position in Architecture Governance Flow

SLGA™ Governance Flow

Software Requirements → Software Creation → Software Dependencies →

Software Configuration → Software Quality → Software Security → Software Release → Software Deployment → Software Change Management → Software Retirement

Cross-cutting:

Software Lifecycle Control

SRLS Core Module Flow

RCMM — Release Candidate Management Module

RRM — Release Readiness Module

RAM — Release Authorization Module

RRMM — Release Record Management Module

RWM — Release Withdrawal Module

RCMM occupies the first Core Module position within SRLS.

It receives relevant software artifact, configuration, dependency, and

lifecycle references established by preceding SLGA™ governance mechanisms.

RCMM establishes the release candidate identity and composition required

for release readiness assessment.

Its principal output is transferred to RRM without implying that readiness has already been established.

Software Artifact References → Candidate Purpose → Candidate Identity →

Candidate Composition → Configuration & Dependency Association → Candidate Boundary → Evidence Review → Candidate Registration → Candidate Classification → Software Release Candidate Profile → Release Readiness Handoff

5\. Governed Space

Software Release Candidate Governance

This governed space concerns the identification, composition, boundaries,

traceability, and controlled lifecycle of software artifacts proposed for release.

It includes:

Candidate identity and registration.

Candidate purpose and scope.

Software product association.

Artifact and build identification.

Candidate composition.

Configuration and dependency references.

Artifact integrity associations.

Candidate completeness.

Candidate modification and replacement.

Candidate version relationships.

Candidate evidence and uncertainty.

Candidate classification.

Candidate historical continuity.

Readiness assessment handoff.

RCMM does not independently establish software correctness, security acceptability, release readiness, or release authorization.

6\. Governed Object

The primary Governed Object is:

A Software Release Candidate State representing the identifiable

relationship between a proposed software release, its constituent artifacts, configuration and dependency baselines, intended scope, relevant lifecycle references, supporting evidence, and current candidate governance conditions.

The governed state must preserve:

Software Product ID

Release Candidate ID

Candidate Purpose

Candidate Scope

Artifact Composition

Build References

Configuration Baseline Reference

Dependency Baseline Reference

Artifact Identity Evidence

Candidate Boundary

Candidate Revision

Candidate Status

Unresolved Conditions

Historical Candidate Reference

Version

7\. Scope

RCMM governs:

Candidate establishment and registration.

Candidate identity and composition.

Candidate scope and boundaries.

Artifact and build association.

Configuration and dependency references.

Candidate creation and assembly context.

Candidate changes and revisions.

Candidate integrity and equivalence references.

Candidate evidence sufficiency.

Candidate classification.

Candidate supersession.

Candidate historical preservation.

Readiness handoff.

RCMM does not independently authorize a release or establish that the proposed software satisfies all applicable requirements.

8\. Foundational Governance Principles

8.1 Candidate Establishment Principle

Software Artifact Available ≠ Release Candidate Established.

8.2 Identity Principle

Version Label Assigned ≠ Release Candidate Identity Verified.

8.3 Composition Principle

Build Artifact Identified ≠ Complete Release Candidate Composition

Established.

8.4 Configuration Principle

Source Version Identified ≠ Executable Configuration Identified.

8.5 Dependency Principle

Dependency Manifest Available ≠ Candidate Dependency State Fully

Established.

8.6 Readiness Boundary Principle

Release Candidate Identified ≠ Release Candidate Ready.

8.7 Authorization Boundary Principle

Release Candidate Registered ≠ Release Authorized.

8.8 Change Integrity Principle

Candidate Modified ≠ Previous Candidate Evidence Automatically Applicable.

8.9 Actor Independence Principle

Automated Candidate Creation ≠ Automated Candidate Governance Established.

8.10 Historical Continuity Principle

Candidate Superseded ≠ Previous Candidate Record Erased.

9\. Release Candidate Identity

Every governed release candidate must establish:

Software Product ID

Release Candidate ID

Candidate Name

Candidate Revision

Candidate Purpose

Candidate Scope

Candidate Creation Reference

Artifact Baseline Reference

Configuration Baseline Reference

Dependency Baseline Reference

Candidate Registration Time

Candidate Status

Version

A candidate identifier must refer to a sufficiently defined candidate state rather than merely an informal software version label.

10\. Candidate Identity Integrity

Release Candidate Name ≠ Verified Release Candidate Identity.

Candidate identity must remain associated with the artifacts and relevant baselines that define the candidate.

11\. Candidate Purpose

The candidate purpose establishes why the software artifact is being proposed for release.

Relevant elements include:

Intended Software Function

Release Objective

Target Software Product

Intended Release Context

Relevant Requirements Baseline

Candidate Purpose Limitations

RCMM records the purpose and relevant references without independently

approving the underlying software requirements.

12\. Purpose Integrity

Release Candidate Purpose Declared ≠ Candidate Behavior Verified Against

That Purpose.

Requirements alignment remains subject to applicable requirements and

readiness assessment mechanisms.

13\. Candidate Scope

Candidate scope defines the software content and intended release

boundaries represented by the candidate.

Relevant elements include:

Included Functionality

Excluded Functionality

Included Components

Included Interfaces

Supported Configurations

Supported Platforms

Intended Distribution Context

Known Scope Limitations

14\. Scope Integrity

Candidate Scope Documented ≠ Candidate Scope Technically Verified.

A declared scope must remain distinguishable from the actual composition and behavior established by supporting evidence.

15\. Software Product Association

RCMM associates each candidate with the software product, service,

application, or system for which release is proposed.

Relevant elements include:

Software Product Identity

Product Baseline

Product Version Context

Release Branch Reference

Relevant Requirements Reference

Existing Release Relationship

Candidate Compatibility Context

16\. Product Association Principle

Candidate Associated With a Product ≠ Candidate Compatible With Every

Product Environment.

17\. Candidate Composition

Candidate composition identifies the artifacts and components represented

by the proposed release.

Relevant elements include:

Executable Artifacts

Application Packages

Container Images

Software Libraries

Required Runtime Components

Configuration Artifacts

Database Migration Artifacts

Interface Definitions

Installation or Distribution Packages

Other Material Release Components

The composition must reflect the applicable release form.

A candidate may consist of one artifact or multiple interdependent artifacts.

18\. Composition Completeness

Primary Executable Identified ≠ Complete Release Candidate Identified.

A release candidate may depend on configuration, migration, runtime, or supporting artifacts that materially affect the resulting software.

19\. Source Artifact References

RCMM preserves relevant source artifact references established through software creation and configuration governance.

Relevant elements include:

Source Repository Reference

Source Revision

Source Branch

Source Baseline

Generated Source References

Relevant Source Change References

Source references support traceability but do not independently establish

executable artifact identity.

20\. Source Identity Principle

Source Revision Equality ≠ Build Artifact Equality.

Identical source revisions may produce materially different artifacts under different build environments, dependencies, or configurations.

21\. Build Artifact References

RCMM associates release candidates with relevant build artifacts.

Relevant elements include:

Build ID

Build Artifact ID

Build Process Reference

Build Time

Build Environment Reference

Build Configuration Reference

Artifact Integrity Reference

Build Evidence Reference

RCMM uses build identity and reproducibility findings established through SCFS.

22\. Build Identity Principle

Successful Build ≠ Identifiable Release Candidate.

Build success alone does not establish candidate composition, scope, or supporting lifecycle associations.

23\. Configuration Baseline Association

RCMM identifies the configuration baseline applicable to the release

candidate.

Relevant elements include:

Configuration Baseline ID

Build Configuration

Runtime Configuration References

Feature Configuration

Environment-Specific Parameters

Configuration Constraints

Configuration Version

Known Configuration Variants

RCMM does not replace configuration baseline establishment under SCFS.

24\. Configuration Integrity

Configuration Declared ≠ Configuration Included in Candidate Verified.

Where configuration materially affects candidate identity or behavior, its applicability must be supported by sufficient evidence.

25\. Dependency Baseline Association

RCMM associates the candidate with relevant dependency governance

references.

Relevant elements include:

Dependency Baseline ID

Direct Dependency References

Transitive Dependency References

Runtime Dependency References

External Service Dependencies

Dependency Version References

Dependency Constraints

Unresolved Dependency Conditions

Dependency acceptance and continuity remain governed by SDS.

26\. Dependency Association Principle

Dependency List Attached ≠ Dependency Baseline Applicable to Candidate

Established.

The relevant dependency state must correspond to the candidate being assessed.

27\. Candidate Assembly

Candidate assembly concerns the formation of the defined release candidate

from its relevant constituent artifacts.

RCMM establishes:

Assembly Event ID

Assembly Purpose

Input Artifact References

Assembly Process Reference

Assembly Actor Context

Assembly Output Reference

Assembly Time

Assembly Evidence

Assembly Exceptions

The assembly process may be manual, automated, or autonomous.

28\. Assembly Integrity

Candidate Assembly Completed ≠ Candidate Composition Correct.

Successful packaging or assembly does not independently establish completeness or suitability.

29\. Artifact Integrity References

RCMM preserves available evidence supporting the identity and integrity of

candidate artifacts.

Relevant mechanisms may include:

Cryptographic Hash References

Artifact Signatures

Immutable Artifact Identifiers

Build Provenance References

Artifact Repository Records

Configuration Evidence

Artifact Comparison Evidence

The selected mechanism must be appropriate to the governed artifact and applicable assurance requirements.

30\. Artifact Integrity Principle

Artifact Hash Available ≠ Complete Candidate Integrity Established.

A hash may support the identity of a particular artifact without establishing that all required artifacts, dependencies, or configurations have been included.

31\. Candidate Boundary

The candidate boundary determines what is included in the governed release candidate.

Relevant dimensions include:

Artifact Boundary

Component Boundary

Configuration Boundary

Dependency Boundary

Functional Scope Boundary

Platform Boundary

Distribution Boundary

Evidence Boundary

Candidate boundaries must remain explicit where their omission could misrepresent the release candidate.

32\. Boundary Integrity

Candidate Boundary Defined ≠ All External Software Reliance Eliminated.

External dependencies and operating conditions may remain relevant even when they are not physically included in the release package.

33\. Candidate Creation Context

RCMM preserves the relevant context in which the release candidate was

established.

Relevant elements include:

Creation Event

Creation Method

Creation Actor

Automated System Reference

Development Environment Reference

Source Baseline

Build Reference

Assembly Reference

Creation Time

Supporting Evidence

34\. Creation Context Principle

Candidate Creation Traceable ≠ Candidate Creation Legitimately Authorized.

General authority legitimacy remains subject to the appropriate

authority-governance mechanisms.

35\. Automated Candidate Creation

RCMM applies when software release candidates are produced by automated build and release preparation systems.

It requires:

Automated Process Identification

Relevant Input References

Generated Artifact Identification

Configuration Association

Dependency Association

Automation Event Traceability

Candidate Registration

Exception Preservation

Automation does not eliminate candidate identity or composition requirements.

36\. Automation Principle

Automated Candidate Generation ≠ Candidate Identity and Composition

Automatically Established.

37\. Autonomous Agent Participation

Where autonomous AI agents participate in candidate preparation, RCMM preserves relevant creation and modification references.

These may include:

Agent Identity Reference

Assigned Development Objective

Artifact Creation Event

Artifact Modification Event

Tool Execution Reference

Build or Assembly Event

Candidate Submission Event

Relevant Authority Reference

Unresolved Actor Context

RCMM governs the software candidate implications of agent activity rather

than general agent behavior.

38\. Agent Participation Principle

Agent Authorized to Develop Software ≠ Agent Authorized to Establish Every

Release Governance Condition.

39\. Candidate Completeness Assessment

Candidate completeness concerns whether the artifacts and references

necessary to identify the proposed release have been sufficiently established.

Relevant dimensions include:

Artifact Coverage

Component Coverage

Configuration Coverage

Dependency Reference Coverage

Build Reference Coverage

Scope Coverage

Required Supporting Artifact Coverage

Known Exclusions

Missing Information

Completeness is assessed against the defined candidate boundary.

40\. Completeness Principle

Candidate Package Present ≠ Candidate Composition Complete.

41\. Candidate Evidence

RCMM may use:

Source Revision Records

Build Records

Artifact Inventories

Artifact Hashes

Configuration Baselines

Dependency Baselines

Assembly Records

Repository Records

Software Creation Records

Automated Pipeline Records

Agent Activity References

Artifact Comparison Results

Evidence must remain associated with the candidate state it supports.

42\. Evidence Provenance

Relevant candidate evidence must preserve:

Evidence ID

Evidence Source

Evidence Origin

Collection or Generation Method

Referenced Artifact

Referenced Candidate

Relevant Event Time

Evidence Creation Time

Evidence Content

Known Limitations

Version

43\. Evidence Integrity

Candidate Evidence Recorded ≠ Candidate Evidence Reliable.

RCMM must preserve the distinction between available evidence and sufficiently supported candidate conclusions.

44\. Evidence Sufficiency

Evidence sufficiency concerns whether the available information adequately

supports candidate identity, composition, boundaries, and relevant baseline associations.

Relevant dimensions include:

Identity Coverage

Artifact Coverage

Composition Coverage

Configuration Coverage

Dependency Association Coverage

Evidence Consistency

Evidence Applicability

Evidence Freshness

Missing Evidence

Conflicting Evidence

45\. Evidence Sufficiency Principle

Release Candidate Documentation Available ≠ Release Candidate Evidence

Sufficient.

46\. Candidate Uncertainty

Candidate uncertainty may arise from:

Unknown Artifact Origin

Incomplete Composition

Missing Build Reference

Unverified Configuration

Unresolved Dependency Association

Conflicting Artifact Identifiers

Uncertain Candidate Scope

Missing Assembly Evidence

Unknown Automated Modification

Uncertainty must remain visible in the candidate governance state.

47\. Uncertainty Principle

Candidate Identity Uncertain ≠ Candidate Identity Established by Assumption.

48\. Candidate Change Management

RCMM governs the candidate-specific consequences of changes occurring after

candidate establishment.

Relevant changes include:

Artifact Replacement

Build Replacement

Configuration Modification

Dependency Modification

Component Addition

Component Removal

Scope Modification

Candidate Reassembly

Generated Artifact Modification

Material changes must be associated with the applicable SCMS and SCFS mechanisms.

49\. Candidate Change Integrity

Candidate Modified ≠ Previous Candidate Evidence Automatically Applicable.

A changed candidate must be reassessed to determine whether previous identity and composition findings remain applicable.

50\. Candidate Revision and Supersession

RCMM distinguishes revisions and replacement candidates.

Relevant elements include:

Original Candidate ID

Revised Candidate ID

Revision Reason

Changed Artifacts

Changed Configuration

Changed Dependencies

Supersession Event

Previous Evidence Applicability

Historical Reference

A superseded candidate must remain reconstructable.

51\. Supersession Principle

Candidate Superseded ≠ Candidate History Erased.

52\. Candidate Equivalence

Candidate equivalence concerns whether two candidate states are sufficiently equivalent for a specified governance purpose.

Relevant dimensions include:

Artifact Identity

Build Identity

Configuration Equivalence

Dependency Equivalence

Functional Scope

Supporting Evidence

Known Differences

Equivalence Boundary

Equivalence must be assessed according to the applicable purpose and conditions.

RCMM does not independently establish universal functional or security equivalence.

53\. Equivalence Principle

Identical Version Labels ≠ Equivalent Release Candidates.

54\. Candidate Registration

Candidate registration establishes the governed reference through which a candidate can be identified and assessed.

At minimum, registration must preserve:

Candidate ID

Software Product ID

Candidate Revision

Candidate Scope

Artifact References

Configuration Baseline Reference

Dependency Baseline Reference

Registration Event

Registration Time

Candidate Classification

Unresolved Conditions

Registration does not establish release readiness.

55\. Registration Principle

Release Candidate Registered ≠ Release Candidate Accepted for Authorization.

56\. Candidate Classification

RCMM may classify candidate states as:

Candidate Proposed

Identification Pending

Identity Established

Composition Pending

Composition Established

Baseline Association Pending

Candidate Registered

Candidate Evidence-Limited

Candidate Incomplete

Candidate Conflicted

Candidate Change Detected

Candidate Reassessment Required

Candidate Superseded

Candidate Withdrawn from Consideration

Candidate Ready for Readiness Assessment

Unknown

These are candidate governance classifications, not release readiness or authorization decisions.

57\. Classification Integrity

Candidate Ready for Readiness Assessment ≠ Release Readiness Satisfied.

A candidate may be sufficiently identified for assessment while remaining technically unsuitable or unauthorized for release.

58\. Candidate Lifecycle Continuity

RCMM preserves candidate identity and history across changes in

participating actors, development systems, repositories, and organizations.

Relevant elements include:

Candidate Identity Continuity

Artifact Reference Continuity

Configuration Reference Continuity

Dependency Reference Continuity

Historical Candidate States

Unresolved Conditions

Evidence Continuity

Custodial Transfer References

Supersession History

Candidate continuity does not independently establish transfer of authority or responsibility.

59\. Continuity Principle

Release Candidate Transferred ≠ Release Governance Authority Automatically

Transferred.

60\. Software Release Candidate Profile

The principal governed output is the:

Software Release Candidate Profile

It establishes the evidence-supported identity, composition, boundaries,

and current governance state of a software artifact or collection of artifacts proposed for release.

The profile preserves:

Software product identity.

Release candidate identity.

Candidate purpose and scope.

Candidate revision.

Artifact composition.

Source and build references.

Configuration baseline.

Dependency baseline.

Assembly context.

Artifact integrity references.

Candidate boundaries.

Supporting evidence.

Evidence limitations.

Candidate changes.

Candidate classification.

Historical references.

Readiness assessment handoff.

61\. Minimum Profile Fields

At minimum:

Software Release Candidate Profile ID

Software Product ID

Release Candidate ID

Candidate Name

Candidate Revision

Candidate Purpose

Candidate Scope

Artifact Composition Register

Source Baseline Reference

Build Artifact Reference

Configuration Baseline Reference

Dependency Baseline Reference

Assembly Event Reference

Artifact Identity Evidence

Candidate Boundary

Candidate Creation Context

Relevant Actor References

Evidence References

Evidence Sufficiency State

Missing Evidence

Evidence Conflicts

Candidate Change References

Candidate Classification

Historical Candidate Reference

Readiness Handoff Status

Effective Time

Version

62\. Minimum Implementation Framework

A minimum operational RCMM implementation should contain eight governed layers.

Layer 1 — Candidate Identity

Establish the software product, candidate identifier, revision, and

candidate purpose.

Layer 2 — Candidate Scope and Boundary

Define the intended release scope, included artifacts, relevant exclusions,

and applicable candidate boundaries.

Layer 3 — Artifact Composition

Identify the constituent artifacts and relevant build references forming

the proposed release candidate.

Layer 4 — Configuration and Dependency Association

Associate the candidate with the applicable configuration and dependency

baselines.

Layer 5 — Candidate Creation and Assembly Context

Preserve relevant creation, assembly, automated process, and actor

references.

Layer 6 — Evidence and Completeness Assessment

Evaluate whether candidate identity, composition, and baseline associations

are sufficiently supported.

Layer 7 — Candidate Classification and Continuity

Establish the candidate state, preserve unresolved conditions, and maintain

revision and supersession history.

Layer 8 — Governed Output and Readiness Handoff

Produce the Software Release Candidate Profile and transfer the established

candidate state to RRM.

63\. Minimum Decision States

At minimum:

Candidate Not Established

Candidate Proposed

Candidate Identification Pending

Candidate Identity Established

Candidate Scope Defined

Candidate Composition Pending

Candidate Composition Established

Configuration Association Pending

Dependency Association Pending

Candidate Baselines Associated

Candidate Evidence Review Pending

Candidate Evidence Sufficient

Candidate Evidence Insufficient

Candidate Incomplete

Candidate Conflicted

Candidate Registered

Candidate Change Detected

Candidate Reassessment Required

Candidate Revised

Candidate Superseded

Candidate Withdrawn from Consideration

Candidate Ready for Readiness Assessment

Readiness Handoff Completed

Unknown

Decision states must not imply release authorization.

64\. Minimum Evidence Path

Where candidate identity or composition cannot be sufficiently established:

Software Product → Proposed Release Candidate → Candidate Identity →

Candidate Scope → Artifact Composition → Build Reference → Configuration Baseline → Dependency Baseline → Assembly Context → Artifact Identity Evidence → Evidence Sufficiency Assessment → Missing / Conflicted / Unknown Condition → Evidence Acquisition or Candidate Correction → Candidate Reassessment → Candidate Classification → Software Release Candidate Profile → Readiness Assessment Handoff

Missing evidence must not be replaced by assumed artifact identity or composition.

65\. Minimum Reassessment Framework

Current Software Release Candidate Profile → New Candidate Event → Artifact

Change Detection → Scope Review → Composition Review → Build Reference Review → Configuration Association Review → Dependency Association Review → Evidence Applicability Review → Candidate Identity Review → Equivalence Assessment Where Required → Candidate Classification Update → Historical State Preservation → Profile Revision → Readiness Handoff Update

Reassessment must distinguish:

Artifact Changed

Configuration Changed

Dependency Changed

Candidate Scope Changed

Candidate Evidence Changed

Candidate Identity Uncertain

Candidate Equivalence Established

Candidate Equivalence Not Established

New Candidate Required

Previous Candidate Superseded

66\. Governance Requirements

A conforming RCMM implementation must satisfy the following requirements.

Candidate Identity and Scope

Identify the software product associated with the candidate.

Establish a distinct release candidate identifier.

Preserve candidate revision information.

Establish candidate purpose.

Define candidate scope.

Identify relevant exclusions and restrictions.

Preserve candidate registration time.

Prevent informal version labels from replacing sufficiently established

candidate identity.

Artifact Composition

Identify material constituent artifacts.

Preserve relevant source references.

Preserve relevant build references.

Identify required supporting artifacts.

Preserve artifact composition relationships.

Identify material composition gaps.

Prevent primary executable identification from automatically establishing

complete candidate composition.

Configuration and Dependencies

Associate the candidate with the applicable configuration baseline.

Preserve relevant configuration variants.

Associate the candidate with the applicable dependency baseline.

Preserve material dependency references.

Identify unresolved baseline associations.

Prevent declared configurations from automatically establishing actual

artifact configuration.

Prevent dependency manifests from automatically establishing complete

dependency reality.

Creation and Assembly

Preserve candidate creation context.

Identify relevant assembly events.

Preserve assembly input references.

Preserve assembly output references.

Preserve automated creation references where applicable.

Preserve relevant autonomous agent participation references.

Distinguish technical creation capability from applicable authority

conditions.

Evidence and Integrity

Preserve candidate evidence references.

Preserve evidence provenance.

Preserve artifact identity evidence.

Evaluate candidate evidence sufficiency.

Identify missing evidence.

Preserve conflicting evidence.

Preserve material uncertainty.

Prevent evidence existence from automatically establishing evidence

reliability.

Prevent artifact hashes from automatically establishing complete

candidate integrity.

Candidate Changes and Continuity

Identify candidate modifications.

Preserve material artifact replacements.

Preserve configuration changes.

Preserve dependency changes.

Preserve candidate scope changes.

Identify changes requiring candidate reassessment.

Preserve candidate revision history.

Preserve supersession relationships.

Prevent material changes from automatically inheriting previous candidate

evidence.

Prevent identical version labels from establishing candidate equivalence.

Preserve historical candidate states.

Classification and Handoff

Establish evidence-supported candidate classifications.

Preserve incomplete candidate states.

Preserve evidence-limited states.

Preserve conflicted states.

Preserve unknown states.

Identify candidates sufficiently established for readiness assessment.

Preserve unresolved candidate conditions during handoff.

Prevent candidate registration from being represented as release

readiness.

Prevent candidate identification from being represented as release

authorization.

Architecture Boundaries

Maintain SRS requirements governance boundaries.

Maintain SCS software creation governance boundaries.

Maintain SDS dependency governance boundaries.

Maintain SCFS configuration and artifact identification boundaries.

Maintain SQS quality governance boundaries.

Maintain SSS security governance boundaries.

Maintain RRM release readiness boundaries.

Maintain RAM release authorization boundaries.

Maintain RRMM release record boundaries.

Maintain SCMS change management boundaries.

Maintain SLCS lifecycle control boundaries.

Produce a Software Release Candidate Profile.

67\. Governance Logic

Canonical RCMM logic:

Software Product Identity → Candidate Purpose → Candidate Scope → Candidate

Identity → Source & Build References → Artifact Composition → Configuration Baseline Association → Dependency Baseline Association → Candidate Assembly Context → Artifact Identity Evidence → Candidate Boundary Review → Completeness Assessment → Evidence Sufficiency Review → Candidate Uncertainty Assessment → Candidate Registration → Candidate Classification → Change & Supersession Review → Historical State Preservation → Software Release Candidate Profile → RRM Readiness Assessment Handoff

Where a material candidate change occurs, RCMM must reassess the applicability of previous candidate identity, composition, and evidence conclusions.

68\. Governance Outputs

RCMM may produce:

Software Release Candidate Profile

Release Candidate Identity Record

Candidate Purpose Record

Candidate Scope Record

Candidate Boundary Record

Candidate Composition Register

Source Artifact Reference Record

Build Artifact Reference Record

Configuration Baseline Association

Dependency Baseline Association

Candidate Assembly Record

Artifact Identity Evidence Record

Candidate Creation Context Record

Automated Creation Reference

Candidate Completeness Assessment

Candidate Evidence Sufficiency State

Missing Evidence Record

Evidence Conflict Record

Candidate Uncertainty Record

Candidate Change Record

Candidate Revision Record

Candidate Supersession Record

Candidate Equivalence Assessment Reference

Candidate Classification

Historical Candidate State Record

Readiness Assessment Handoff

Version

69\. Practical Applications

AI-Generated Applications

Identifying exact release candidates created by AI coding systems and

preserving their artifact, build, configuration, and dependency references.

Autonomous Software Factories

Maintaining candidate identity and composition when multiple autonomous agents continuously generate and modify software artifacts.

Enterprise Software Releases

Establishing consistent candidate registration, composition control, and

readiness handoff across development teams and products.

Continuous Integration Pipelines

Distinguishing successful builds from governed release candidates and

ensuring that candidate evidence applies to the correct artifacts.

Containerized Applications

Associating container images, configuration references, runtime

dependencies, and supporting deployment artifacts with an identifiable release candidate.

Critical Software Systems

Preserving release candidate traceability where artifact identity and configuration consistency are essential to subsequent assurance and authorization.

70\. Use Case 1 — An Autonomous Software Factory Produces Multiple

Candidates Under One Version Label

An organization operates an autonomous software development environment.

Several AI agents independently modify application code, generate tests,

update dependencies, and assemble software packages.

The automated pipeline produces three successive build artifacts within a short period.

Each artifact carries the same visible version label:

Version 2.4.0

The first artifact contains the original dependency baseline.

The second includes an updated external library.

The third includes a modified configuration and an additional software

function.

The automated system reports that all three builds completed successfully.

A release agent selects the latest artifact and submits it for release

readiness assessment.

However, the candidate registration record identifies only the version

label and does not establish which exact build, dependency baseline, or configuration is being submitted.

RCMM evaluates:

Candidate Identity → Build Artifact Identity → Artifact Composition →

Configuration Baseline → Dependency Baseline → Candidate Evidence → Candidate Classification

The assessment identifies:

Three materially distinguishable build artifacts exist.

All artifacts carry the same version label.

Their dependency and configuration conditions differ.

The proposed release candidate is not sufficiently identified by its version label.

Supporting evidence may apply to different artifacts.

Candidate identity must be established before readiness assessment can reliably proceed.

The appropriate candidate governance classification is:

Candidate Identity Unresolved — Readiness Handoff Not Established.

The critical distinctions are:

Version Label Equality ≠ Artifact Identity.

Build Success ≠ Candidate Identification.

Latest Artifact Selected ≠ Release Candidate Governed.

The result: RCMM prevents an automated release process from treating a

shared version label as sufficient proof of candidate identity.

71\. Use Case 2 — A Release Candidate Omits a Required Database

Migration Artifact

A software team prepares a release candidate for an enterprise application.

The application introduces changes requiring an updated database schema.

The application package is successfully assembled.

The build completes without errors.

The primary executable is registered as the release candidate.

However, the required database migration artifact is not included in the candidate composition record.

The development team assumes that the migration will be handled automatically during deployment.

No sufficient evidence establishes which migration artifact belongs to the

proposed release or whether it corresponds to the application's expected database state.

RCMM evaluates:

Candidate Scope → Required Artifact Composition → Supporting Artifacts →

Configuration Association → Composition Completeness → Evidence Sufficiency

The assessment identifies:

The primary application artifact is identifiable.

The software requires a database schema change.

The associated migration artifact has not been sufficiently identified.

Candidate composition is incomplete for the defined release scope.

The release candidate cannot be represented as compositionally

established.

The appropriate classification is:

Candidate Incomplete — Required Artifact Association Missing.

The critical distinctions are:

Application Package Complete ≠ Release Candidate Complete.

Migration Expected ≠ Migration Artifact Identified.

Build Success ≠ Candidate Composition Established.

The result: RCMM exposes a missing release component before incomplete

candidate identity is transferred into release readiness assessment.

72\. Critical Distinctions

Software Artifact Available ≠ Release Candidate Established.

Version Label Assigned ≠ Release Candidate Identity Verified.

Release Candidate Name ≠ Verified Release Candidate Identity.

Candidate Scope Documented ≠ Candidate Scope Technically Verified.

Candidate Associated With a Product ≠ Candidate Compatible With Every

Product Environment.

Primary Executable Identified ≠ Complete Release Candidate Identified.

Source Revision Equality ≠ Build Artifact Equality.

Successful Build ≠ Identifiable Release Candidate.

Configuration Declared ≠ Configuration Included in Candidate Verified.

Dependency List Attached ≠ Dependency Baseline Applicable to Candidate

Established.

Candidate Assembly Completed ≠ Candidate Composition Correct.

Artifact Hash Available ≠ Complete Candidate Integrity Established.

Candidate Creation Traceable ≠ Candidate Creation Legitimately Authorized.

Automated Candidate Generation ≠ Candidate Identity and Composition

Automatically Established.

Candidate Package Present ≠ Candidate Composition Complete.

Candidate Evidence Recorded ≠ Candidate Evidence Reliable.

Candidate Identity Uncertain ≠ Candidate Identity Established by Assumption.

Candidate Modified ≠ Previous Candidate Evidence Automatically Applicable.

Candidate Superseded ≠ Candidate History Erased.

Identical Version Labels ≠ Equivalent Release Candidates.

Release Candidate Registered ≠ Release Candidate Accepted for Authorization.

Candidate Ready for Readiness Assessment ≠ Release Readiness Satisfied.

Release Candidate Transferred ≠ Release Governance Authority Automatically

Transferred.

73\. Cross-Architecture Boundary

RCMM may consume governed references from:

SRS — Software Requirements Standard

SCS — Software Creation Standard

SDS — Software Dependency Standard

SCFS — Software Configuration Standard

SQS — Software Quality Standard

SSS — Software Security Standard

SCMS — Software Change Management Standard

SLCS — Software Lifecycle Control Standard

Within SRLS, RCMM provides the identifiable release candidate to:

RRM — Release Readiness Module

RAM — Release Authorization Module

RRMM — Release Record Management Module

RWM — Release Withdrawal Module

RCMM governs release candidate identity, composition, boundaries, and

continuity.

It does not independently establish requirements correctness, dependency acceptance, configuration validity, software quality, security suitability, release readiness, or authorization.

Where material changes require reassessment of an upstream governance condition, the responsible standard retains authority over that condition.

74\. Cross-Architecture Mechanism Principle

An operational mechanism may produce inputs for multiple governance architectures, but it belongs only to the architecture whose governed domain includes that mechanism itself. Other architectures govern only its domain-specific relevance or consequences.

RCMM may reference software creation events, dependency records, configuration baselines, build artifacts, quality findings, and security evidence.

It governs their relevance to release candidate identity and composition.

It does not redefine or replace the mechanisms that establish those underlying governance states.

75\. Canonical Boundary

RCMM governs how software artifacts are identified, assembled, distinguished, bounded, registered, maintained, and presented as release candidates under evidence-supported identity, composition, configuration, dependency, and lifecycle conditions.

It does not independently determine software correctness, authorize software release, authorize deployment, certify security, or establish general actor authority.

Its principal governed output is the Software Release Candidate Profile.

76\. Canonical Closing Statement

A software release cannot be governed if the software being proposed for

release cannot be sufficiently identified.

Modern software may be assembled from source code, generated components, external libraries, configuration files, container images, runtime dependencies, and supporting artifacts.

Automated pipelines can produce these artifacts rapidly.

Autonomous agents can modify them continuously.

Version labels may remain unchanged while the underlying software changes materially.

A successful build may therefore represent a technically functioning artifact without establishing a sufficiently identifiable release candidate.

RCMM establishes the first governance condition of software release: the

candidate itself must be identifiable, bounded, compositionally traceable, and supported by applicable evidence before its readiness can be meaningfully assessed.

A release candidate is not governed because it has a name, version number, package, or successful build. It is governed when the exact software proposed for release can be distinguished from alternatives, associated with its relevant baselines, and reconstructed through its supporting lifecycle records.

RCMM transforms software release preparation from artifact selection into

governed candidate establishment — ensuring that every subsequent readiness assessment and authorization decision refers to the software that is actually proposed for release, rather than an assumed or ambiguously identified version.